Autodesk®
Scaleform®

Scaleform

Scaleform 3.1 C++ Flash ActionScript

Mustafa Thamer

2.01

2010 9 27

Autodesk®
GAMEWARE

Copyright Notice

Autodesk® Scaleform® 4.4

Disclaimer

Scaleform

MD 20770

6305 310

Scaleform

www.scaleform.com

info@scaleform.com

(301) 446-3200

(301) 446-3199

1 C++ Flash ActionScript

Flash® ActionScript™

ActionScript Flash

ActionScript

Flash Autodesk Scaleform® Flash

ActionScript C++ C++

ActionScript ActionScript

C++ Flash

ActionScript ÆC++	C++ ÆActionScript
FSCommand	GFx:Movie::Get/SetVariable ActionScript
ExternalInterface	GFx:Movie::Invoke ActionScript
Direct Access API Uses GFx::Value	

1.1 ActionScript C++

Scaleform C++ ActionScript FSCommand ExternalInterface
ActionScript API Flash FSCommand
ExternalInterface Scaleform C++ Scaleform
GFx::Loader GFx::MovieDef GFx::Movie
Scaleform Scaleform documentation.

FSCommand ActionScript 'fscommand' fscommand ActionScript
C++
C++ fscommand ActionScript
fscommand

ExternalInterface ActionScript flash.external.ExternalInterface.call
ActionScript C++ ExternalInterface
ActionScript GFx::Value C++ ExternalInterface
GFx::Value ActionScript ExternalInterface.call ActionScript
ExternalInterface API Scaleform
ActionScript Flash

FSCommands ExternalInterface

Direct Access API GFx::FunctionHandler FSCommands ExternalInterface
ActionScript VM C++ ActionScript
C++ GFx::FunctionHandler Direct
Access API

1.1.1 FSCommand

ActionScript fscommand ActionScript
```
fscommand("setMode", "2");
```

fscommand

ActionScript fscommand GFx FSCommand
GFx::FSCommandHandler fscommand GFx::Loader
GFx::MovieDef GFx::Movie

GFx::Loader -> GFx::MovieDef -> GFx::Movie

GFx::RenderConfig EdgeAA GFx::Translator

GFx::Loader GFx::Movie

FSCommand GFxPlayerTiny (

“FxPlayerFSCommandHandler”)

fscommand GFx::FSCommandHandler

```
class    OurFSCommandHandler : public FSCommandHandler
{
   public:
       virtual void Callback(Movie* pmovie, const char* pcommand,
                             const char* parg)
   {
      Printf("FSCommand: %s, Args: %s", pcommand, parg);
   }
};
```

ActionScript fscommand

fscommand fscommand

GFx::Loader

```
// Register our fscommand handler
Ptr<FSCommandHandler> pcommandHandler = *new OurFSCommandHandler;
gfxLoader-    >SetFSCommandHandler(pcommandHandler);
```

GFx::Loader GFx::Movie GFx::MovieDef

SetFSCommandHandler

C++

Advance Invoke calls

1.1.2 API

Flash ExternalInterface. fscommand

ExternalInterface fscommand

ExternalInterface

```
class OurExternalInterfaceHandler : public ExternalInterface
{
   public:
      virtual void Callback(Movie* pmovieView, const char* methodName,
                            const Value* args, unsigned                    argCount)
   {
         Printf("ExternalInterface: %s, %d args: ", methodName, argCount);
         for(unsigned      i = 0; i < argCount; i++)
         {
               switch(args[i].GetType())
               {
                  case     Value::VT_Number:
                        Printf("%3.3f", args[i].GetNumber());
                        break;
                  case     Value::VT_String:
                        Printf("%s", args[i].GetString());
                        break;
               }
               Printf("%s", (i == argCount -          1) ? "" : ", ");
         }
         Printf("\      n");

         // return a value of 100
         Value retValue;
        retValue.SetNumber(100);
         pmovieView-   >SetExternalInterfaceRetVal(retValue);
    }
};
```

GFx::Loader

```
Ptr<ExternalInterface> pEIHandler = *new OurExternalInterfaceHandler;
gfxLoader.SetExternalInterface(pEIHandler);
```

ActionScript ExternalInterface

1.1.2.1 FxDelegate

ExternalInterface ExternalInterface

methodName

methodName

GFx::Value FxDelegate (
Apps\Samples\GameDelegate\FxGameDelegate.h

FxDelegate GFx::ExternalInterface /
methodName

FxDelegate HUD Kit FxDelegate minimap
minimap FxDelegate

```
// Minimap Begin
void FxPlayerApp::Accept( FxDelegateHandler::CallbackProcessor* cbreg)
{
  cbreg- >Process( "registerMiniMapView", FxPlayerApp::RegisterMiniMapView);

  cbreg- >Process( "enableSimulation", FxPlayerApp::EnableSimulation);
  cbreg- >Process( "changeMode", FxPlayerApp::ChangeMode);
  cbreg- >Process( "captureStats", FxPlayerApp::CaptureStats);

  cbreg- >Process( "loadSettings", FxPlayerApp::LoadSettings);
  cbreg- >Process( "numFriendliesChange", FxPlayerApp::NumFriendliesChange);
  cbreg- >Process( "numEnemiesChange", FxPlayerApp::NumEnemiesChange);
  cbreg- >Process( "numFlagsChange", FxPlayerApp::NumFlagsChange);
  cbreg- >Process( "numObjectivesChange", FxPlayerApp::NumObjectivesChange);
  cbreg- >Process( "numWaypointsChange", FxPlayerApp::NumWaypointsChange);
  cbreg- >Process( "playerMovementChange", FxPlayerApp::PlayerMovementChange);
  cbreg- >Process( "botMovementChange", FxPlayerApp::BotMovementChange);

  cbreg- >Process( "showingUI", FxPlayerApp::ShowingUI);
}
```

FxDelegate

1.1.2.2 ActionScript

ActionScript

```
import flash.external.*;
ExternalInterface.call("foo", arg);
```

“foo” “arg” 0
ExternalInterface API Flash

ExternalInterface ExternalInterface

flash.external.ExternalInterface.call("foo",arg)

1.1.2.3 ExternalInterface.addCallback

ExternalInterface.addCallback ActionScript C++
C++
GFx::Movie::Invoke()

AS Code:

```
import     flash.external.*;

var txtField:TextField = this.createTextField("txtField",
                         this.getNextHighestDepth(), 0, 0, 200, 50);

varaliasName:String = "setText";
var instance:Object = txtField;
var method:Function = SetText;
var wasSuccessful:Boolean = ExternalInterface.addCallback(aliasName, instance,
                                    method);
function SetText()
{
        trace(this + ".SetText ");
        this.text = "INVOKED!";
}
```

SetText ActionScript "this" txtField

C++ Code:

```
pMovie-  >Invoke("setText", "");
```

"txtField.SetText" "INVOKED!" "txtField"
GFx::Movie::Invoke() 1.2.2 .

1.2 C++ ActionScript

ActionScript C++ C++ Flash
Scaleform Scaleform C++ ActionScript
ActionScript

Direct Access API C++ ActionScript
Direct Access API

1.2.1 ActionScript

Scaleform GetVariable and SetVariable, ActionScript
Direct Access API 2
Set/GetVariable Direct Access API
Direct Access call GFx::Value::SetMember
GetVariable ActionScript
GFx::Value Direct Access

GetVariable SetVariable ActionScript

```
int counter = (int)pHUDMovie -          >GetVariableDouble("_root.counter");
counter++;
pHUDMovie- >SetVariable("_root.counter", GFx::Value((double)counter));
```

GetVariableDouble _root.counter C++ Double
GetVariableDouble 0. SetVariable
_root.counter GFx::Movie GetVariable SetVariable

SetVariable GFx::Movie::SetVarType “ ”
Flash t _root.mytextfield
SetVariable(“_root.mytextfield.text”, “testing”, SV_Normal) 1
SV_Sticky ( )
_root.mytextfield.text 3
C++ SV_Normal SV_Sticky
SV_Normal

Scaleform SetVariableArray GetVariableArray.

SetVariableArray

GFx::Movie::SetVariableArraySize

SetVariableArray AS

```
int idxInASArray = 0;
const char* strarr[2];
strarr[0] = "This is the first string";
strarr[1] = "This is the second one";
pMovie-  >SetVariableArray(Movie::SA_String, "_root.strArray",
                idxInASArray, strarr, 2);
```

```
int idxInASArray = 0;
const wchar_t* strarr[2];
strarr[0] = L"This is the first string";
strarr[1] = L"This is the second one";
pMovie-  >SetVariableArray(Movie::SA_StringW, "_root.strArray",
                idxInASArray, strarr, 2)
```

The GetVariableArray AS

1.2.2 ActionScript

ActionScript ActionScript GFx::Movie::Invoke()

UI UI

Direct Access API

ActionScript

_root level “mySlider” SetSliderPos "mySlider"

AS Code:

```
this.SetSliderPos = function (pos)
{
    // Clamp the incoming position value
    if (pos<rangeMin) pos = rangeMin;
    if (pos>rangeMax) pos = rangeMax;
    gripClip._x = trackClip._width * ((pos-                rangeMin)/(rangeMax-       rangeMin));
    gripClip.gripPos = gripClip._x;
}
```

"gripClip" "mySlider" pos rangeMin/Max

C++ Code:

```
Value result;
bool bInvoked = pMovie-        >Invoke("_root.mySlider.SetSliderPos", &result, "%d",
                        newPos);
```

Invoke

ActionScript

ActionScript Scaleform log

ActionScript GFx::Movie::Advance

GFx::MovieDef::CreateInstance initFirstFrame 1

ActionScript

GFx::Movie::IsAvailable() AS

```
Value result;
if (pMovie-      >IsAvailable("parentPath.mySlider.SetSliderPos"))
pMovie-        >Invoke("parentPath.mySlider.SetSliderPos", &result, "%d", newPos);
```

'printf' Invoke

GFx::Value

```
Value args[3], result;
args[0].SetNumber(i);
args[1].SetString("test");
args[2].SetNumber(3.5);
pMovie-  >Invoke("path.to.methodName", &result, args, 3);
```

InvokeArgs Invoke va_list

Invoke InvokeArgs printf vprintf

1.2.3

Flash

GFx::Movie

Movie::IsAvailable(path)
Movie::SetVariable(path, value)
Movie::SetVariableDouble(path, value)
Movie::SetVariableArray(path, index, data, count)
Movie::SetVariableArraySize(path, count)
Movie::GetVariable(value, path)
Movie::GetVariableDouble(path)
Movie::GetVariableArray(path, index, data, count)
Movie::GetVariableArraySize(path)
Movie::Invoke(pathToMethodName, result, argList, ...)

“.”

"clip2" "clip1"

Main Stage -> clip1 -> clip2

1. clip1 /
2. clip2 clip1

"clip1.clip2"
"_root.clip1.clip2"
"_level0.clip1.clip2"

"Clip1.clip2" <- - "Clip1"
"clip2" <- "clip1." -

"_root" "_level0" ActionScript 2
ActionScript 3
ActionScript 3 :_root _level0 root level0
SWF _root _levelN ( )

- Flash Studio Test Movie (Ctrl-Enter) (Ctrl-Alt-V)
( )

- 1
(Ctrl-Alt-V) ( )

- loadMovie
_level0 _level1 _level2 ( )
_levelN.objectName
_levelN.objectPath.objectName N

1. Paths to Objects and Variables Jesse Stratford
2. Advanced Pathing Jesse Stratford

2 Direct Access API

Scaleform 3.1 Direct Access AS ActionScript API

UI Direct Access HUD Kit Minimap Demo minimap Flash

Direct Access GFx::Values ActionScript Display Objects Display Objects MovieClips Buttons TextFields GFx::Value API GFx::Value

Direct Access API C++ (GFx::Value) ActionScript ActionScript GFx::Movie AS AS (Direct Access)

GFx::Movie ActionScript (AS) GFx::Value 'foo' Invoke Invoke

1. GFx::Movie Invoke ( ):

```
Value ret;
Value args[N];
pMovie->Invoke("_root.foo.method", &ret, args, N);
```

2. Direct Access Invoke ( ):

```
(Assumes 'foo' holds a reference to an AS object at "_root.foo")
Value ret;
Value args[N];
bool = foo.Invoke("method", &ret, args, N);
```

Invoke AS Direct Access API GFx::Value::HasMember GetMember SetMember.

GetMember MovieClip (GFx::Value)

```
Value    tf;
MovieClip.GetMember("textField", &tf);
tf.SetText("hello");
```

MovieClip textfield SetText

2.1

Direct Access API AS
C++

```
Movie* pMovie = … ;
Value parent, child;
pMovie-  >CreateObject(&parent);
pMovie-  >CreateObject(&child);
parent.SetMember("      child"    , child);
```

CreateObject ActionScript

```
Value mat, params[6];
// (                    params[0..6] ) …
pMovie-  >CreateObject(&mat, "flash.geom.Matrix", params, 6);
```

```
// (          pMovie          GFx::Movie*):
Value owner, childArr, childObj;
pMovie-  >CreateArray(&owner);                    //
pMovie-  >CreateArray(&childArr);                 //
pMovie-  >CreateObject(&childObj);                //
bool = owner.SetElement(0, childArr);             //      parent[0]
bool = owner.SetElement(1, childObj);             //      parent[1]
…
bool = foo.SetMember("owner", owner); //                          foo        'owner'
```

GFx::Value::VisitMembers() ObjectVisitor
Visit DisplayObject

VisitMembers

ActionScript ASSetPropFlags

e.g.: _global.ASSetPropFlags(MyClass.prototype, ["someFunc", "__get__number", "test"], 6, 1);

Properties (getter/setter) VisitMembers ASSetPropFlags

Direct Access VT_Object "[ ]" VT_Object ( ) "[ ]"

2.2

Direct Access API Object DisplayObject

MovieClips Buttons TextFields DisplayInfo API

DisplayInfo API DisplayObject

SetMember SetDisplayInfo

GFx::Movie::SetVariable

SetDisplayInfo movieclip

```
// MovieClip GFx::Value*
Value::DisplayInfo info;
PointF pt(100,100);
info.SetRotation(90);
info.SetPosition(pt.x, pt.y);
MovieClip.SetDisplayInfo(info);
```

2.3

ActionScript2 (AS2 VM)

GFx::Movie::CreateFunction C++

CreateFunction

AS C++

fscommand ExternalInterface

AS

```
Value    obj;
pmovie-  >GetVariable(&obj,          "_root.obj");

class    MyFunc  :  public     FunctionHandler
{
     public:
          virtual      void    Call(   const    Params&   params)
          {
                  //              /
          }
};
Ptr<  MyFunc>  customFunc    = *SF_HEAP_NEW(Memory::GetGlobalHeap())          MyFunc();
Value    func;
pmovie-  >CreateFunction(&func,          customFunc);
obj.   SetMember(  "func",       func);
```

ActionScript (_root timeline)

```
obj.func(param1, param2, param3);
```

Call() Params :

- pRetVal GFx::Value , GFx::Value
  , pMovie->CreateObject() or CreateArray() ( ),

- pMovie
- pThis ,
- ArgCount
- pArgs GFx::Values [ ]
  GFx::Value firstArg = params.pArgs[0];
- pArgsWithThisRef (
  ,)
- pUserData

```
Value      networkProto,      networkCtorFn,      networkConnectFn;
class      NetworkClass    :  public     FunctionHandler
{
 public:
  enum       Method
```

```
    {
        METHOD_Ctor,
        METHOD_Connet,
    };

    virtual             ~NetworkClass() {}
    virtual             void    Call(   const    Params&   params)
    {
        int             method   =  int(    params.  pUserData);
        switch              ( method)
        {
        case            METHOD_Ctor:
            //
            break;
        case            METHOD_Connet:
            //
            break;
        }
    }
};
    Ptr<  NetworkClass>     networkClassDef       = *SF_HEAP_NEW(Memory::GetGlobalHeap())
                                        NetworkClass();
    //
    pmovie-  >CreateFunction(&networkCtorFn,            networkClassDef,
                    (                   void*)NetworkClass::METHOD_Ctor);
    //
    pmovie-  >CreateObject(&networkProto);
    //
    networkCtorFn.     SetMember(  "prototype",         networkProto);
    //        “         ”
    pmovie-  >CreateFunction(&networkConnectFn,             networkClassDef,
                    (                   void*)NetworkClass::METHOD_Connet);
    networkProto.      SetMember(  "connect",       networkConnectFn);
    //              _global
pmovie-      >SetVariable(      "_global.Network",          networkCtorFn);



    var netObj:Object = new Network(param1, param2);



Value       origFuncReal;
obj.       GetMember(  "funcReal", &origFuncReal);
```

```
class FuncRealIntercept : public FunctionHandler
{
  Value OrigFunc;
  public:
    FuncRealIntercept( Value origFunc) : OrigFunc( origFunc) {}
    virtual ~FuncRealIntercept() {}
    virtual void Call( const Params& params)
    {
      //
      OrigFunc. Invoke( "call", params. pRetVal, params. pArgsWithThisRef,
      params. ArgCount + 1);
      //
    }
};
Value funcRealIntercept;
  Ptr< FuncRealIntercept> funcRealDef = *SF_HEAP_NEW(Memory::GetGlobalHeap())
                    FuncRealIntercept( origFuncReal);
pmovie- >CreateFunction(&funcRealIntercept, funcRealDef);
obj. SetMember( "funcReal", funcRealIntercept);
```

GFx::Value

.swf (GFx::Movie) GFx::Values

2.4 Direct Access

Direct Access API GFx::Value

2.4.1 Object

	('foo' Object "_root.foo")
	bool has = foo. HasMember("bar");
	Value bar; bool = foo. GetMember ("bar", &bar);
	Value val; bool = foo. SetMember ("bar", val);
	Value ret; Value args[N]; bool = foo.Invoke("method", args, N, &ret); or foo. Invoke ("method", args, N);

	Value obj;
	pMovie- >CreateObject(&obj);
	…
	bool = foo.SetMember("bar", obj);

	Value owner, child;
	pMovie- >CreateObject(&owner);
	pMovie- >CreateObject(&child);
	bool = owner.SetMember("child", child);
	…
	bool = foo. SetMember ("owner", owner);

	Value::ObjectVisitor v; foo.VisitMembers(&v);
	bool = foo. DeleteMember ("bar");

2.4.2 Array

	('bar' Array "_root.foo.bar" 'foo' Object)
	unsigned sz = bar. GetArraySize ();
	Value val; bool = bar.GetElement(idx, &val);
	Value val; bool = bar.SetElement(idx, val);
	bool = bar. SetArraySize (N);

	Value arr;
	pMovie- >CreateArray(&arr);
	…
	bool = foo.SetMember("bar", arr);

Complex Object	Value owner, childArr, childObj;
	pMovie- >CreateArray(&owner);
	pMovie- >CreateArray(&childArr);
	pMovie- >CreateObject(&childObj);
	bool = owner.SetElement(0, childArr);
	bool = owner.SetElement(1, childObj);
	…
	bool = foo.SetMember("owner", owner);

	for (UPInt i=0; i <N; i++) { … bool = bar.GetElement(i+idx, &val) … } : Value::ArrayVisitor v;

	bar. VisitElements (v, idx, N);
	bool = bar. ClearElements ();

	PushBack
	GValue val;
	bool = bar.PushBack(val);
	
	PopBack
	Value val;
	bool = bar.PopBack(&val); or void bar.PopBack();

	bool = bar.RemoveElement(idx);
	
	bool = bar.RemoveElements(idx, N);

2.4.3

	('foo' "_root.foo.bar" (MovieClip, TextField, Button))
	Value::DisplayInfo info; bool = foo.GetDisplayInfo(&info);
	Value::DisplayInfo info; … bool = foo.SetDisplayInfo(info);
	Matrix2 _3 mat; bool = foo. GetDisplayMatrix (&mat);
	Matrix2 _3 mat; … bool = foo.SetDisplayMatrix(mat);
	Render::Cxform cxform; bool = foo.GetColorTransform(&cxform);
	Render ::Cxform cxform; … bool = foo.SetColorTransform(cxform);

2.4.4

	('foo' "_root.foo")
	Value newInstance; bool = foo.AttachMovie(&newInstance, "SymbolName", "instanceName");
	Value emptyInstance; bool = foo.CreateEmptyMovieClip(&emptyInstance, "instanceName");
	bool = foo. GotoAndPlay ("myframe");
	bool = foo. GotoAndPlay (3);
	bool = foo. GotoAndStop ("myframe");
	bool = foo. GotoAndStop (3);

2.4.5

	('foo' "_root.foo")
	Value text; bool = foo. GetText (&text);
	Value text; … bool = bar.SetText(text);
HTML	Value htmlText; bool = bar.GetTextHTML (&htmlText);
HTML	Value htmlText; … bool = bar. SetTextHTML (htmlText);